

Arabic Distress Alarm — CNN vs Band-Limited Spectral Entropy Baseline

Ready-to-run notebook for Google Colab.

Instructions:

1. Upload your dataset to [/content/dataset/disaster](#) and [/content/dataset/non_disaster](#).
2. Run cells from top to bottom (0 → 14).
3. First run without augmentation, then try with augmentation to test robustness.

```
# ♦ Cell 0 – Install & Imports
!pip -q install librosa==0.10.1 soundfile==0.12.1

import os, glob, json
import numpy as np
import librosa, soundfile as sf

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, f1_score, roc_auc_score
from sklearn.utils.class_weight import compute_class_weight
from sklearn.linear_model import LogisticRegression

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print('TensorFlow:', tf.__version__)
print('Librosa:', librosa.__version__)
print('✅ Imports ready')
```

```
TensorFlow: 2.19.0
Librosa: 0.10.1
✅ Imports ready
```

```
# ♦ Cell 1 – Paths & Global Config
DATA_DIR = "/content/dataset" # نوافذ 1 ثانية لو بتستخدمين Google Drive

# تأكدني أسماء المجلدات:
classes = ['disaster', 'non_disaster']

SR = 16_000
TARGET_SECONDS = 1.0 # نوافذ 1 ثانية ←
TARGET_SAMPLES = int(SR * TARGET_SECONDS)

# Log-Mel features
N_FFT = 1024
HOP_LENGTH = 160 # ~10ms
WIN_LENGTH = 400 # 25ms
N_MELS = 64
FMIN, FMAX = 80, 7600

RNG = np.random.default_rng(42)
USE_AUG = False # لتحسين المتانة True أولاً، ثم False جزئي

print('✅ Config set')
```

```
✅ Config set
```

```
# ♦ Cell 2 – Check folders & counts
for sub in classes:
    p = os.path.join(DATA_DIR, sub)
    cnt = len(glob.glob(os.path.join(p, "*")))
    print(f"{sub}<14> exists? {os.path.exists(p)} | count = {cnt}")
    if not os.path.exists(p):
        raise FileNotFoundError(f"❌ Folder not found: {p} – add audio files")
```

```
disaster      exists? True | count = 21
non_disaster  exists? True | count = 29
```

```
# ♦ Cell 3 – Raw & fixed loaders
def load_audio_raw(fp, sr=SR):
    # بدون قص للطول (mono + resample) يقرأ الملف كامل
    y, _ = librosa.load(fp, sr=sr, mono=True)
    if np.max(np.abs(y)) > 0:
        y = y / np.max(np.abs(y))
```

```

return y

def load_audio_fixed(fp, sr=SR, target_samples=TARGET_SAMPLES):
    # هذه تُستخدم فقط عندما نريد طولاً ثابتاً (Val/Test)
    y = load_audio_raw(fp, sr=sr)
    if len(y) < target_samples:
        y = np.pad(y, (0, target_samples - len(y)))
    else:
        y = y[:target_samples]
    return y

print('✅ load_audio_raw & load_audio_fixed ready')

```

✅ load_audio_raw & load_audio_fixed ready

```

# ♦ Cell 4 – Optional Augmentation (safe version, no direct time_stretch)
def augment_audio(y, sr=SR):
    # 1) ضوضاء بيضاء خفيفة
    if RNG.random() < 0.7:
        noise = RNG.normal(0, 1, size=y.shape)
        y = y + 0.03 * np.std(y) * noise

    # 2) تغيير طبقة 2± نصف نغمة
    if RNG.random() < 0.5:
        steps = int(RNG.choice([-2, -1, 1, 2]))
        try:
            y = librosa.effects.pitch_shift(y, sr=sr, n_steps=steps)
        except Exception:
            pass

    # 3) تمطيط زمني "بديل آمن":
    # ثم نصحح الطبقة عشوائياً ما تتغير resample نمزج/نبتئ الصوت عبر
    if RNG.random() < 0.5:
        rate = float(RNG.uniform(0.9, 1.1)) # 0.9-1.1
        try:
            # غير السرعة بإعادة التعيين (يرفع/ينزل الطبقة مؤقتاً)
            y_fast = librosa.resample(y, orig_sr=sr, target_sr=int(sr*rate))
            # رجع الطول المستهدف
            if len(y_fast) >= TARGET_SAMPLES:
                y_fast = y_fast[:TARGET_SAMPLES]
            else:
                y_fast = np.pad(y_fast, (0, TARGET_SAMPLES-len(y_fast)))
            # صحح تغيير الطبقة عكسياً (12-تقريباً*log2(rate))
            corr_steps = -12.0 * np.log2(rate)
            y = librosa.effects.pitch_shift(y_fast, sr=sr, n_steps=corr_steps)
        except Exception:
            # لو صار خطأ، نتجاهل الجزء ونكمل بدون تمطيط
            pass

    # تطبيع نهائي
    if np.max(np.abs(y)) > 0:
        y = y / np.max(np.abs(y))
    return y

```

```

# ♦ Cell 5 – To Log-Mel
def to_logmel(y, sr=SR, n_fft=N_FFT, hop=HOP_LENGTH, win_length=WIN_LENGTH, n_mels=N_MELS, fmin=FMIN, fmax=FMAX):
    S = librosa.feature.melspectrogram(
        y=y, sr=sr, n_fft=n_fft, hop_length=hop, win_length=win_length,
        n_mels=n_mels, fmin=fmin, fmax=fmax, power=2.0
    )
    S = librosa.power_to_db(S, ref=np.max)
    S = (S - S.min()) / (S.max() - S.min() + 1e-9)
    return S.astype(np.float32)

print('✅ to_logmel ready')

```

✅ to_logmel ready

```

# ♦ Cell 6 – Build file list & labels
files, labels = [], []
for idx, sub in enumerate(classes):
    for fp in glob.glob(os.path.join(DATA_DIR, sub, "*")):
        if os.path.isfile(fp):
            files.append(fp)
            labels.append(idx)
files = np.array(files)
labels = np.array(labels)

```

```
print("Total files:", len(files), "| Class dist.:",
      {classes[i]: int(np.sum(labels==i)) for i in range(len(classes))})
```

```
Total files: 50 | Class dist.: {'disaster': 21, 'non_disaster': 29}
```

```
# ♦ Cell 7 – Stratified split (70/15/15)
X_train, X_tmp, y_train, y_tmp = train_test_split(
    files, labels, test_size=0.30, random_state=42, stratify=labels
)
X_val, X_test, y_val, y_test = train_test_split(
    X_tmp, y_tmp, test_size=0.50, random_state=42, stratify=y_tmp
)
print("Train:", len(X_train), " Val:", len(X_val), " Test:", len(X_test))
```

```
Train: 35 Val: 7 Test: 8
```

```
# ♦ Cell 8 – Windowed loading (train uses RAW, val/test use central FIXED window)
def extract_windows(y, win=TARGET_SAMPLES, step=None):
    if step is None:
        step = win // 2 # 50% overlap
    out = []
    # لو الملف أطول من نافذة واحدة، نقطع بنوافذ متداخلة
    if len(y) >= win:
        for s in range(0, len(y)-win+1, step):
            seg = y[s:s+win]
            out.append(seg)
    else:
        # ملف أقصر من النافذة: نكملة بالصفر
        out = [np.pad(y, (0, win-len(y)))]
    return out

def load_batch_mode(file_list, mode="train"):
    X = []
    for fp in file_list:
        if mode == "train":
            # نقرأ الملف كامل ونقطعه إلى نوافذ
            y = load_audio_raw(fp)
            wins = extract_windows(y, win=TARGET_SAMPLES, step=TARGET_SAMPLES//2)
            for seg in wins:
                if USE_AUG:
                    seg = augment_audio(seg)
                lm = to_logmel(seg)
                X.append(lm[None, ...]) # [1, n_mels, T]
        else:
            # Val/Test: نافذة مركزية واحدة ثابتة الطول
            y = load_audio_raw(fp)
            start = max(0, (len(y)-TARGET_SAMPLES)//2)
            seg = y[start:start+TARGET_SAMPLES]
            if len(seg) < TARGET_SAMPLES:
                seg = np.pad(seg, (0, TARGET_SAMPLES-len(seg)))
            lm = to_logmel(seg)
            X.append(lm[None, ...])
    X = np.stack(X, axis=0).astype(np.float32) if len(X)>0 else np.zeros((0,1,N_MELS,1), np.float32)
    return X

# حقل الميزات
USE_AUG = True # Aug، ايدني بدون
X_train_arr = load_batch_mode(X_train, mode="train")
X_val_arr = load_batch_mode(X_val, mode="val")
X_test_arr = load_batch_mode(X_test, mode="test")

# ليتوافق مع عدد نوافذ التدريب ابني y_train_expanded
def windows_count_for_files(file_list):
    counts = []
    for fp in file_list:
        y = load_audio_raw(fp)
        c = len(extract_windows(y, win=TARGET_SAMPLES, step=TARGET_SAMPLES//2))
        counts.append(max(1, c))
    return counts

train_counts = windows_count_for_files(X_train)
y_train_expanded = np.concatenate([np.full(c, lbl) for c, lbl in zip(train_counts, y_train)]).astype(np.float32)

# y_val/y_test (ملف واحد ← نافذة واحدة)
y_val_f = y_val.astype(np.float32)
y_test_f = y_test.astype(np.float32)

# Keras expects [B, H, W, C]
X_train_k = np.transpose(X_train_arr, (0,2,3,1))
X_val_k = np.transpose(X_val_arr, (0,2,3,1))
X_test_k = np.transpose(X_test_arr, (0,2,3,1))
```

```

X_train_k = np.transpose(X_train_arr, (0,2,3,1))
X_test_k = np.transpose(X_test_arr, (0,2,3,1))

print("Train windows:", X_train_k.shape, "| Val:", X_val_k.shape, "| Test:", X_test_k.shape)
print("y_train_expanded:", y_train_expanded.shape)

```

```

Train windows: (1104, 64, 101, 1) | Val: (7, 64, 101, 1) | Test: (8, 64, 101, 1)
y_train_expanded: (1104,)

```

```

# ♦ Cell 9 – CNN model
def make_cnn(input_shape):
    inp = keras.Input(shape=input_shape)
    x = layers.Conv2D(32, (3,3), padding="same")(inp)
    x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
    x = layers.MaxPool2D((2,2))(x)

    x = layers.Conv2D(64, (3,3), padding="same")(x)
    x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
    x = layers.MaxPool2D((2,2))(x)

    x = layers.Conv2D(128, (3,3), padding="same")(x)
    x = layers.BatchNormalization()(x); x = layers.ReLU()(x)

    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.2)(x)
    out = layers.Dense(1, activation="sigmoid")(x)
    model = keras.Model(inp, out)
    return model

model = make_cnn(X_train_k.shape[1:])
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        keras.metrics.Precision(name="precision"),
        keras.metrics.Recall(name="recall"),
    ],
)
model.summary()

```

Model: "functional_7"

Layer (type)	Output Shape	Param #
input_layer_7 (InputLayer)	(None, 64, 101, 1)	0
conv2d_21 (Conv2D)	(None, 64, 101, 32)	320
batch_normalization_21 (BatchNormalization)	(None, 64, 101, 32)	128
re_lu_21 (ReLU)	(None, 64, 101, 32)	0
max_pooling2d_14 (MaxPooling2D)	(None, 32, 50, 32)	0
conv2d_22 (Conv2D)	(None, 32, 50, 64)	18,496
batch_normalization_22 (BatchNormalization)	(None, 32, 50, 64)	256
re_lu_22 (ReLU)	(None, 32, 50, 64)	0
max_pooling2d_15 (MaxPooling2D)	(None, 16, 25, 64)	0
conv2d_23 (Conv2D)	(None, 16, 25, 128)	73,856
batch_normalization_23 (BatchNormalization)	(None, 16, 25, 128)	512
re_lu_23 (ReLU)	(None, 16, 25, 128)	0
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None, 128)	0
dropout_7 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 1)	129

Total params: 93,697 (366.00 KB)
Trainable params: 93,249 (364.25 KB)

```

# ♦ Cell 10 – Class weights & Training
cw = compute_class_weight(class_weight='balanced',
                           classes=np.unique(y_train_expanded.astype(int))

```

```

x_train_k, y_train_k = x_train_expanded.astype(int), y_train_expanded.astype(int)
y=y_train_expanded.astype(int))

class_weight = {0: cw[0], 1: cw[1]}
print("Class weights:", class_weight)

callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2)
]

history = model.fit(
    X_train_k, y_train_expanded,
    validation_data=(X_val_k, y_val_f),
    epochs=30, batch_size=8,
    class_weight=class_weight,
    callbacks=callbacks, verbose=1
)

```

Class weights: {0: np.float64(6.133333333333334), 1: np.float64(0.5443786982248521)}

```

Epoch 1/30
138/138 ————— 19s 139ms/step - accuracy: 0.9824 - loss: 0.0594 - precision: 0.9953 - recall: 0.9854 - val_accuac
Epoch 2/30
138/138 ————— 22s 160ms/step - accuracy: 0.9877 - loss: 0.0671 - precision: 0.9976 - recall: 0.9889 - val_accuac
Epoch 3/30
138/138 ————— 20s 142ms/step - accuracy: 0.9971 - loss: 0.0402 - precision: 1.0000 - recall: 0.9968 - val_accuac
Epoch 4/30
138/138 ————— 20s 148ms/step - accuracy: 0.9883 - loss: 0.0476 - precision: 0.9987 - recall: 0.9887 - val_accuac
Epoch 5/30
138/138 ————— 19s 139ms/step - accuracy: 0.9779 - loss: 0.0849 - precision: 0.9944 - recall: 0.9814 - val_accuac
Epoch 6/30
138/138 ————— 21s 141ms/step - accuracy: 0.9861 - loss: 0.0421 - precision: 0.9998 - recall: 0.9850 - val_accuac
Epoch 7/30
138/138 ————— 21s 149ms/step - accuracy: 0.9863 - loss: 0.0629 - precision: 0.9961 - recall: 0.9890 - val_accuac

```

♦ Cell 11 – Evaluation + Threshold tuning from Val

لتعظيم Val احسبني أفضل عتبة من F1

y_prob_val = model.predict(X_val_k, verbose=0).ravel()

best_t, best_f1 = 0.5, -1.0

for t in np.linspace(0.2, 0.8, 25):

try:

f1_t = f1_score(y_val_f, (y_prob_val >= t).astype(int))

except Exception:

f1_t = -1

if f1_t > best_f1:

best_f1, best_t = f1_t, t

print(f"Best threshold from Val = {best_t:.3f} (F1={best_f1:.3f})")

طَبِّق العتبة الأفضل على الاختبار

y_prob = model.predict(X_test_k, verbose=0).ravel()

y_pred = (y_prob >= best_t).astype(int)

المقاييس

try:

auc = roc_auc_score(y_test_f, y_prob)

except ValueError:

auc = float('nan')

نحسبها يدويًا أفضل هنا (اختياري) Keras دقة/براعة/استرجاع من

f1 = f1_score(y_test_f, y_pred)

acc = (y_pred == y_test_f).mean()

print(f"✅ Test – Acc: {acc*100:.2f}% | F1: {f1:.3f} | ROC-AUC: {auc:.3f}\n")

print("Classification Report:\n", classification_report(y_test_f, y_pred, target_names=classes, digits=3, zero_division=0))

print("Confusion Matrix:\n", confusion_matrix(y_test_f, y_pred))

Best threshold from Val = 0.200 (F1=1.000)

✅ Test – Acc: 87.50% | F1: 0.909 | ROC-AUC: 0.667

Classification Report:

	precision	recall	f1-score	support
disaster	1.000	0.667	0.800	3
non_disaster	0.833	1.000	0.909	5
accuracy			0.875	8
macro avg	0.917	0.833	0.855	8
weighted avg	0.896	0.875	0.868	8

Confusion Matrix:

```
[[2 1]
 [0 5]]
```

```

# ♦ Cell 12 – Improved Baseline (Windowed features + Scaling + Val threshold)
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# --- ميزات مساعدة سريعة ---
def feat_entropy(y, sr=SR, fmin=FMIN, fmax=FMAX, n_fft=N_FFT, hop=HOP_LENGTH, win_length=WIN_LENGTH):
    S = np.abs(librosa.stft(y, n_fft=n_fft, hop_length=hop, win_length=win_length))**2
    freqs = librosa.fft_frequencies(sr=sr, n_fft=n_fft)
    band = (freqs >= fmin) & (freqs <= fmax)
    P = S[band, :] + 1e-12
    P = P / np.sum(P, axis=0, keepdims=True)
    H = -np.sum(P * np.log(P), axis=0) # entropy لكل إطار
    return H # نرجع الإطارات، ونجمع لاحقاً

def feat_centroid(y, sr=SR, n_fft=N_FFT, hop=HOP_LENGTH, win_length=WIN_LENGTH):
    C = librosa.feature.spectral_centroid(y=y, sr=sr, n_fft=n_fft, hop_length=hop, win_length=win_length)[0]
    return C

def feat_rolloff(y, sr=SR, n_fft=N_FFT, hop=HOP_LENGTH, win_length=WIN_LENGTH, roll=0.85):
    R = librosa.feature.spectral_rolloff(y=y, sr=sr, roll_percent=roll, n_fft=n_fft, hop_length=hop, win_length=win_length)[0]
    return R

def feat_zcr(y, hop=HOP_LENGTH):
    frame_len = 2*hop
    Z = librosa.feature.zero_crossing_rate(y=y, frame_length=frame_len, hop_length=hop)[0]
    return Z

def summarize(vec):
    # نحصل من المتجه الإطارى إحصائيات بسيطة
    v = np.asarray(vec)
    return np.array([
        np.mean(v), np.std(v), np.median(v),
        np.percentile(v, 10), np.percentile(v, 90),
        np.max(v)
    ], dtype=np.float32)

def window_features_for_file(fp):
    # نوافذ 1 ثانية متداخلة 50% (نفس التدريب)
    y_full = load_audio_raw(fp)
    wins = extract_windows(y_full, win=TARGET_SAMPLES, step=TARGET_SAMPLES//2)
    feats = []
    for seg in wins:
        H = summarize(feat_entropy(seg, sr=SR))
        C = summarize(feat_centroid(seg, sr=SR))
        R = summarize(feat_rolloff(seg, sr=SR))
        Z = summarize(feat_zcr(seg))
        f = np.concatenate([H, C, R, Z], axis=0) # 24 (4*6) بعد تجميعي
        feats.append(f)
    return np.stack(feats, axis=0) # [num_windows, feat_dim]

# --- بناء مصفوفات السمات ---
def build_set(files_list):
    X_list, y_list, file_index = [], [], []
    for i, (fp, lbl) in enumerate(zip(files_list, (y_train if files_list is X_train else (y_val if files_list is X_val else y_test)))):
        F = window_features_for_file(fp)
        X_list.append(F)
        y_list.append(np.full(F.shape[0], lbl, dtype=np.int32))
        file_index.extend([i]*F.shape[0])
    X = np.vstack(X_list)
    y = np.concatenate(y_list)
    file_index = np.array(file_index)
    return X, y, file_index

# --- Baseline ---
Xb_tr, yb_tr, tr_file_idx = build_set(X_train)
Xb_va, yb_va, va_file_idx = build_set(X_val)
Xb_te, yb_te, te_file_idx = build_set(X_test)

# Scaling
scaler = StandardScaler()
Xb_tr_s = scaler.fit_transform(Xb_tr)
Xb_va_s = scaler.transform(Xb_va)
Xb_te_s = scaler.transform(Xb_te)

# مصنف بسيط
blr = LogisticRegression(max_iter=2000, class_weight='balanced')
blr.fit(Xb_tr_s, yb_tr)

# احتمالات النوافذ
p_va_win = blr.predict_proba(Xb_va_s)[:,-1]
p_te_win = blr.predict_proba(Xb_te_s)[:,-1]

```

```
# يعمل كويس max/percentile → أصوات الاستغاثة قصيرة) "تجمع من "نوافذ" إلى "ملف"
def pool_by_file(p_win, file_idx, mode='max'):
    outs = []
    for i in np.unique(file_idx):
        q = p_win[file_idx==i]
        if mode == 'max':
            outs.append(np.max(q))
        elif mode == 'mean':
            outs.append(np.mean(q))
        else: # percentile-90
            outs.append(np.percentile(q, 90))
    return np.array(outs, dtype=np.float32)

p_va_file = pool_by_file(p_va_win, va_file_idx, mode='max')
p_te_file = pool_by_file(p_te_win, te_file_idx, mode='max')

# لبيات الملفات (index ملف واحد لكل) لبيات الملفات
yb_va_file = np.array([y_val[i] for i in np.unique(va_file_idx)], dtype=np.int32)
yb_te_file = np.array([y_test[i] for i in np.unique(te_file_idx)], dtype=np.int32)

# CNN (مثال الـ) F1 لتعظيم Val نضبط العتبة من
best_t_bl, best_f1_bl = 0.5, -1.0
for t in np.linspace(0.2, 0.8, 25):
    f1_t = f1_score(yb_va_file, (p_va_file >= t).astype(int), zero_division=0)
    if f1_t > best_f1_bl:
        best_f1_bl, best_t_bl = f1_t, t

# (ملفياً) Test نقيم على
y_pred_file = (p_te_file >= best_t_bl).astype(int)
f1_bl_file = f1_score(yb_te_file, y_pred_file, zero_division=0)
auc_bl_file = roc_auc_score(yb_te_file, p_te_file) if len(np.unique(yb_te_file))>1 else float('nan')

print(f" ♦ Improved Baseline – Thr(Val)={best_t_bl:.3f} | F1(Test-file): {f1_bl_file:.3f} | ROC-AUC(Test-file): {auc_bl_file:.3f}")
print("Confusion Matrix (file-level):\n", confusion_matrix(yb_te_file, y_pred_file))
```

```
♦ Improved Baseline – Thr(Val)=0.325 | F1(Test-file): 0.833 | ROC-AUC(Test-file): 0.667
Confusion Matrix (file-level):
[[1 2]
 [0 5]]
```

```
# ♦ Cell 13 – Save model & labels
SAVE_DIR = "/content/artifacts"
os.makedirs(SAVE_DIR, exist_ok=True)
model.save(os.path.join(SAVE_DIR, "cnn_arabic_distress.h5"))
with open(os.path.join(SAVE_DIR, "label_map.json"), "w", encoding="utf-8") as f:
    json.dump({"0": classes[0], "1": classes[1]}, f, ensure_ascii=False)
print("Saved to:", SAVE_DIR)
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format Saved to: /content/artifacts

```
# ♦ Cell 14 – Quick single-file inference
# جزئي أي ملف من مجلداتك:
TEST_FILE = X_test[0] # أو غيريه لمسار محدد
y = load_audio_fixed(TEST_FILE)
lm = to_logmel(y)[None, ..., None] # [1, n_mels, T, 1]
lm_k = np.transpose(lm, (0,2,1,3))

# Val استخدمتي الأفضل اللي حسيناها من
pred_prob = float(model.predict(lm_k, verbose=0).ravel()[0])
pred_lbl = classes[int(pred_prob >= best_t)]
print(f"File: {TEST_FILE}\nProb(distress)={pred_prob:.3f} | Pred={pred_lbl} | Thr={best_t:.3f}")
```

File: /content/dataset/non_disaster/AUDIO-2025-10-24-02-19-06.wav
Prob(distress)=0.848 | Pred=non_disaster | Thr=0.200

```
# 📊 Cell 15 – Training Curves
import matplotlib.pyplot as plt

hist = history.history

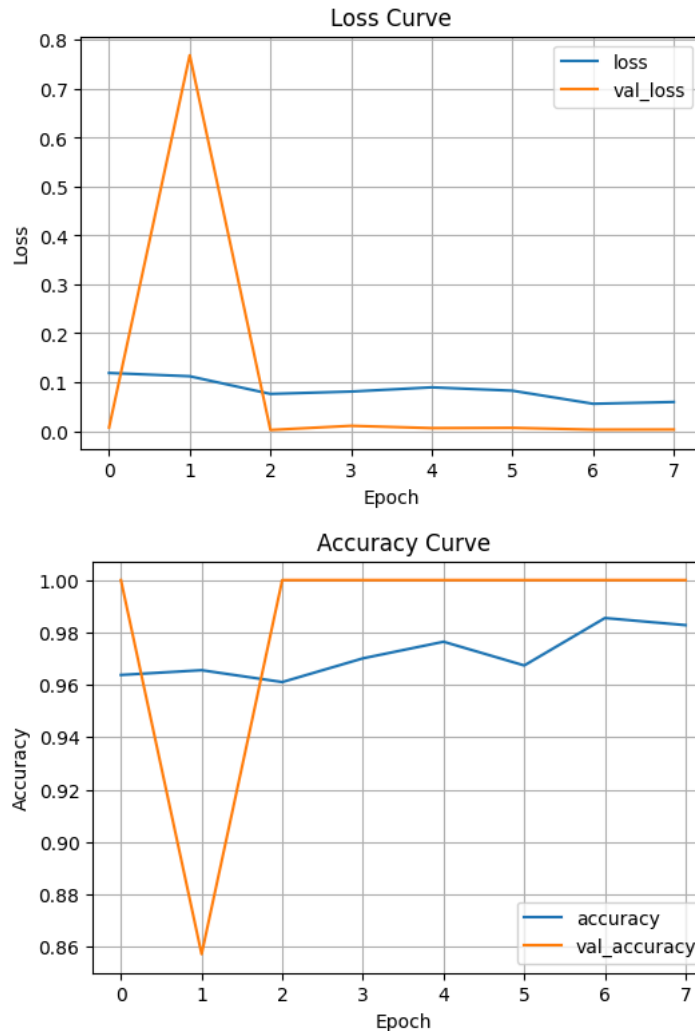
def plot_curve(key, title=None, ylabel=None):
    if key not in hist or ("val_"+key) not in hist: # يرسم فقط إذا المتركين موجودين
        print(f"skip: {key} (not found)")
        return
    plt.figure(figsize=(6,4))
    plt.plot(hist[key], label=key)
    plt.plot(hist["val_"+key], label="val "+key)
```

```

plt.title(title or key)
plt.xlabel("Epoch")
plt.ylabel ylabel or key)
plt.grid(True)
plt.legend()
plt.show()

plot_curve("loss", "Loss Curve", "Loss")
plot_curve("accuracy", "Accuracy Curve", "Accuracy")
plot_curve("auc", "ROC-AUC Curve", "AUC") # يظهر فقط لو ضفتي compile

```



skip: auc (not found)

```

# Cell 16 - Confusion Matrix Heatmap (matplotlib)
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test_f, y_pred)
fig, ax = plt.subplots(figsize=(4,4))
im = ax.imshow(cm, interpolation='nearest')
ax.figure.colorbar(im, ax=ax)

ax.set(xticks=np.arange(len(classes)),
      yticks=np.arange(len(classes)),
      xticklabels=classes, yticklabels=classes,
      xlabel='Predicted label', ylabel='True label',
      title='Confusion Matrix (Test)')

# أرقام داخل الخلايا
th = cm.max() / 2.0 if cm.max()>0 else 0.5
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, format(cm[i, j], 'd'),
              ha="center", va="center",
              color=("white" if cm[i, j] > th else "black"))
plt.tight_layout()
plt.show()

```

